



Lab 2 — Git Intermediate

Mục tiêu: Mô phỏng workflow team thực tế — Gitflow, resolve conflict, rebase, stash, PR workflow.

Phần 1 — Gitflow Branch Strategy

Bước 1: Setup cấu trúc Gitflow

```
mkdir team-project && cd team-project
git init
echo "v1.0.0" > version.txt
git add . && git commit -m "chore: init project"

git checkout -b develop    # branch tích hợp chính
```



Bước 2: Giả lập 2 developer làm song song

```
# Dev A làm login
git checkout -b feature/login
echo "login page" > login.html
echo "Welcome" >> README.md
git add . && git commit -m "feat: add login page"

# Dev B làm dashboard (bắt đầu từ develop)
git checkout develop
git checkout -b feature/dashboard
echo "dashboard" > dashboard.html
echo "Hello World" >> README.md      # cùng sửa README → sẽ conflict!
git add . && git commit -m "feat: add dashboard"
```

Phần 2 — Resolve Merge Conflict

Bước 3: Merge feature/login vào develop trước

```
git checkout develop
git merge feature/login
```

Bước 4: Merge feature/dashboard — gặp conflict

```
git merge feature/dashboard
# CONFLICT (content): Merge conflict in README.md
```

File README.md lúc này trông như sau:

```
<<<<<<< HEAD
Welcome
=====
Hello World
>>>>>>> feature/dashboard
```

Bước 5: Giải quyết conflict

```
# Sửa README.md thủ công: giữ cả hai hoặc chọn 1
# Ví dụ giữ cả hai:
echo -e "Welcome\nHello World" > README.md

git add README.md
git commit -m "merge: resolve conflict in README.md"
```

💡 Trong VS Code có giao diện "Accept Current / Incoming / Both Changes" rất tiện.

Phần 3 — Rebase để giữ lịch sử sạch

Bước 6: Tạo feature mới, dùng rebase thay merge

```
git checkout -b feature/profile develop
echo "profile page" > profile.html
git add . && git commit -m "feat: add profile page"

# Giả lập: develop có thêm commit mới trong lúc đó
git checkout develop
echo "v1.1.0" > version.txt && git add . && git commit -m "chore: bump version"

# Thay vì merge, dùng rebase để "nối" lên đầu develop
git checkout feature/profile
git rebase develop
```

Trước rebase:
develop: A-B-C-D
feature: A-B-X-Y

Sau rebase:
develop: A-B-C-D
feature: D-X'-Y'

Bước 7: Interactive Rebase — gộp commit nhỏ lại

```
# Thêm vài commit lặt vặt
echo "fix1" >> profile.html && git add . && git commit -m "fix: typo"
echo "fix2" >> profile.html && git add . && git commit -m "fix: css indent"

# Gộp 3 commit thành 1 trước khi merge
git rebase -i HEAD~3
# Trong editor: đổi "pick" → "squash" (s) cho commit 2, 3
# → Lịch sử sạch hơn khi mở PR
```

Phần 4 — Git Stash (xử lý tình huống khẩn)

Bước 8: Đang làm dở thì có bug production

```
# Đang làm feature, chưa commit được
echo "work in progress" > wip.txt
git add wip.txt

# Sếp gọi: "Bug gấp trên production!" → cất tạm lại
git stash push -m "wip: feature profile đang làm dở"

# Checkout sang main, tạo hotfix
git checkout main
git checkout -b hotfix/critical-bug
echo "patched" > bugfix.txt
git add . && git commit -m "fix: patch critical production bug"
git checkout main && git merge hotfix/critical-bug

# Xong rồi, quay lại làm tiếp
git checkout feature/profile
git stash pop # lấy lại công việc đang dở
```


Phần 5 — Pull Request Workflow

Bước 9: Chuẩn bị branch trước khi mở PR

```
# Sync với develop mới nhất trước khi push
git fetch origin
git rebase origin/develop

# Kiểm tra những gì sẽ vào PR
git diff develop..feature/profile
git log develop..feature/profile --oneline

# Push lên remote
git push origin feature/profile
# → Mở Pull Request trên GitHub/GitLab, request review
```

Bước 10: Sau khi PR được approve

```
git checkout develop
git merge --no-ff feature/profile -m "merge: feature/profile
into develop"
# --no-ff: luôn tạo merge commit → dễ trace lịch sử team

# Dọn dẹp branch
git branch -d feature/profile
git push origin --delete feature/profile
```